EDS Lab Assignments

Practical1

Name: Harshal Dhanait

PRN: 202501040049

1. Calculate Momentum

Problem Statement:

Write a programthataccepts the mass of an object (in kilograms) and its velocity (in meters per second), thencalculates and displays the momentum of the object. The momentum is calculated usingtheformula: p = m * v
where:
m is the mass oftheobject (in kilograms).
v is the velocityoftheobject (in meters per second).

Input Format:

- A singlefloating-point number representing the mass of the object in kilograms.
- A singlefloating-point number representing the velocity of the object in meters per second.

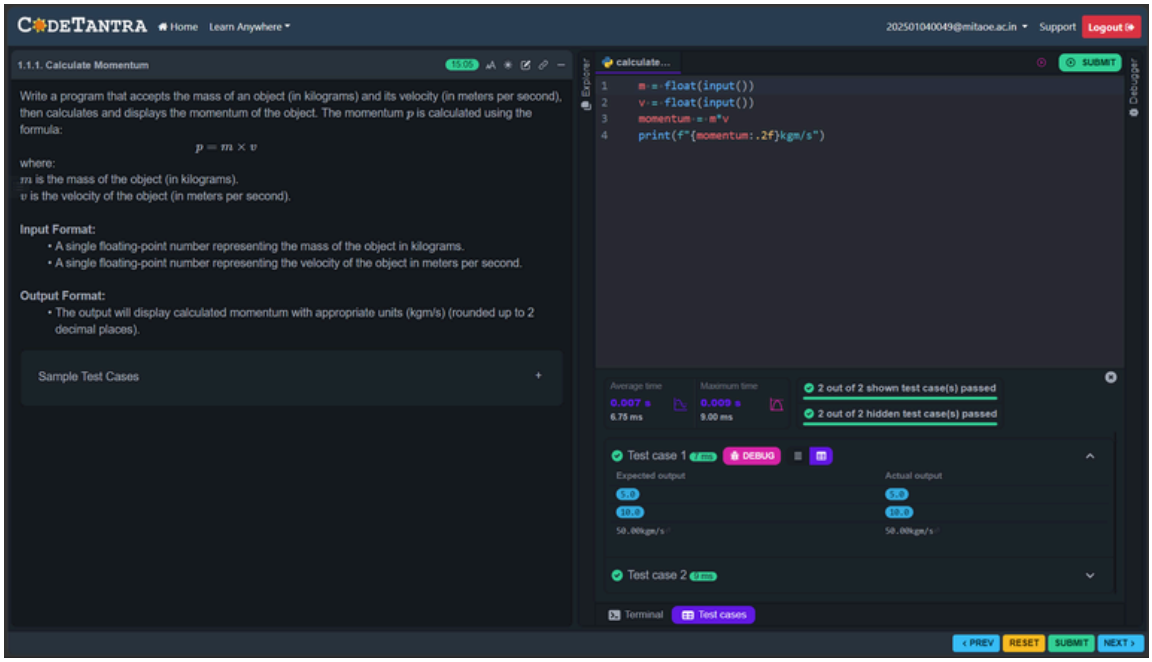
Output Format:

- Theoutputwilldisplay calculated momentum with appropriate units (kgm/s) (roundedupto2decimal places).

Code:

```
mass = float(input())
velocity = float(input())
momentum= mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:



2. Conditional Calculation based on the Number of Digits

Problem Statement:
WriteaPythonprogram that accepts an integer as input. Depending on the number of digits in.
Constraints: 1≤n≤ 999

Input Format:

- The inputconsists of a single integer.

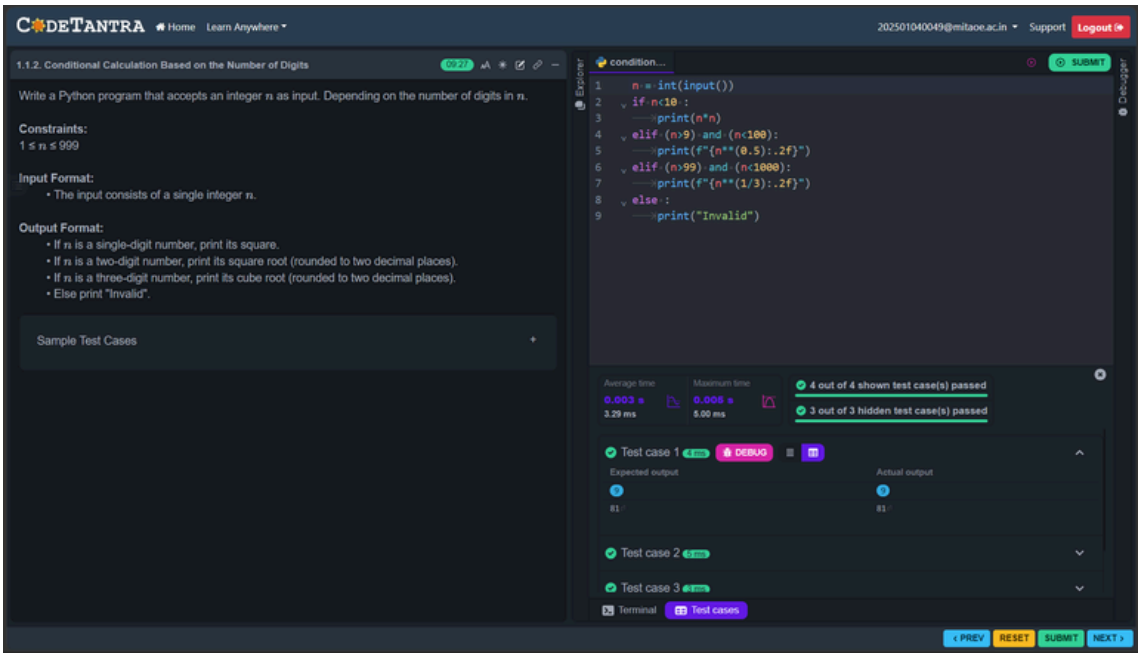
Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```
importmath
n=int(input())
sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
        print(f"{sq}")
elif n>=10 and n<=99:
        print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
        print(f"{cb_root:.2f}")
else:
        print("Invalid")
```

Output:



3. Days Between Two Dates

Problem Statement:
Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

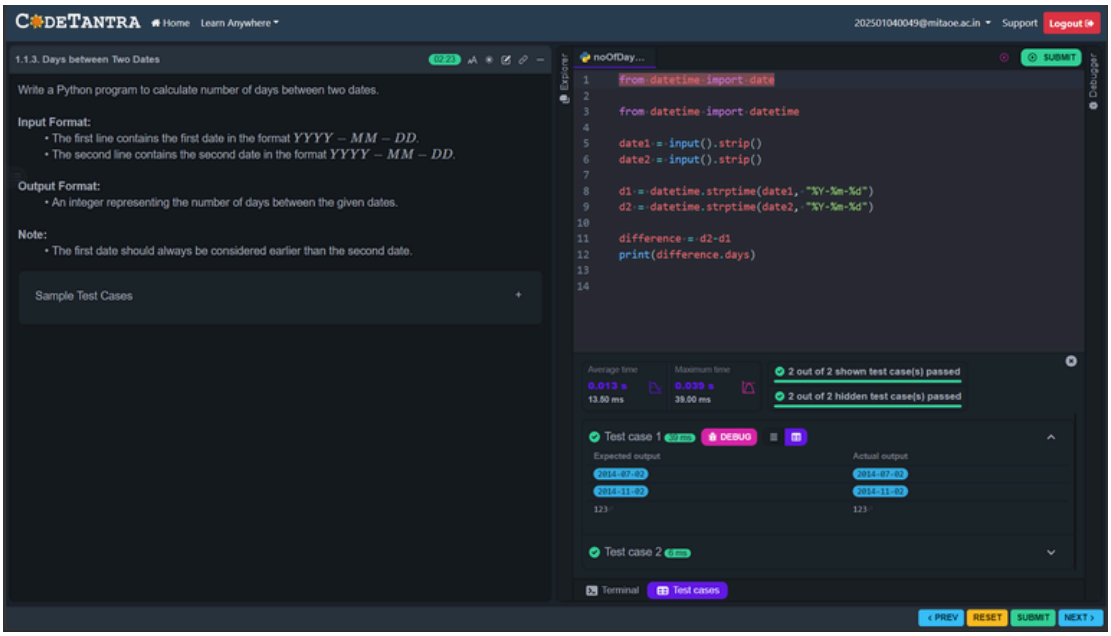
Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:



4. Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

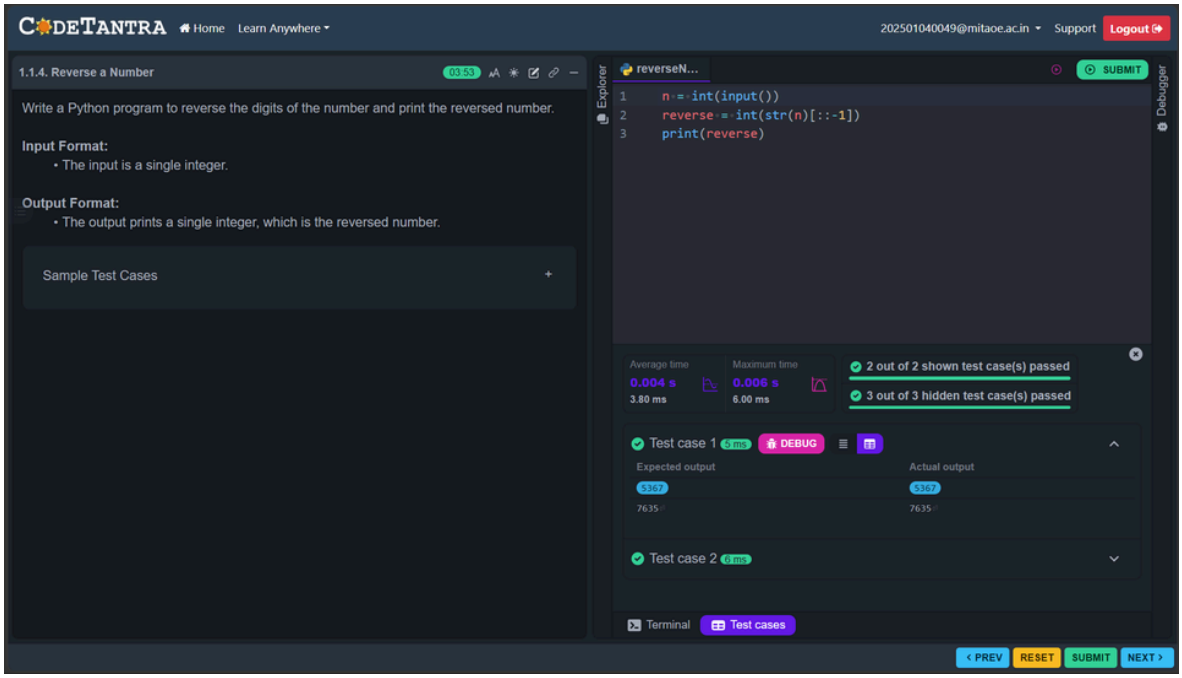
Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

Output:



5. Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

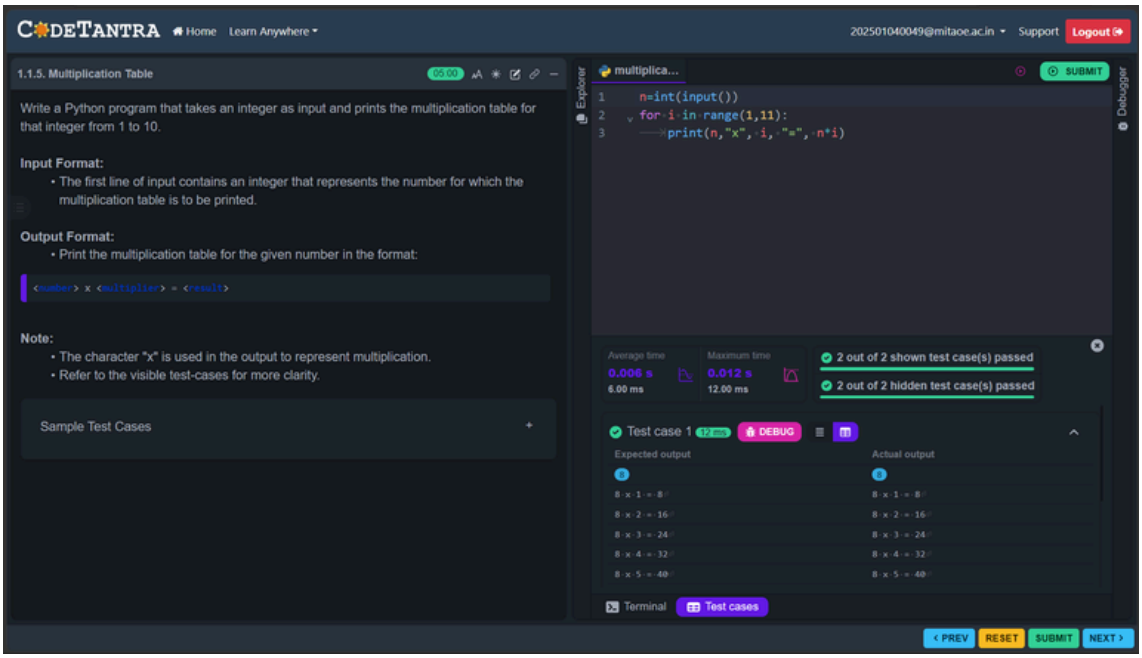
- Print the multiplication table for the given number in the format:
  <**number**> x <**multiplier**> = <**result**>

Code:

```
n = int(input())

for i in range(1,11):
        print(n, "x", i, "=", n*i)
```

Output:



1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.
The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:
If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

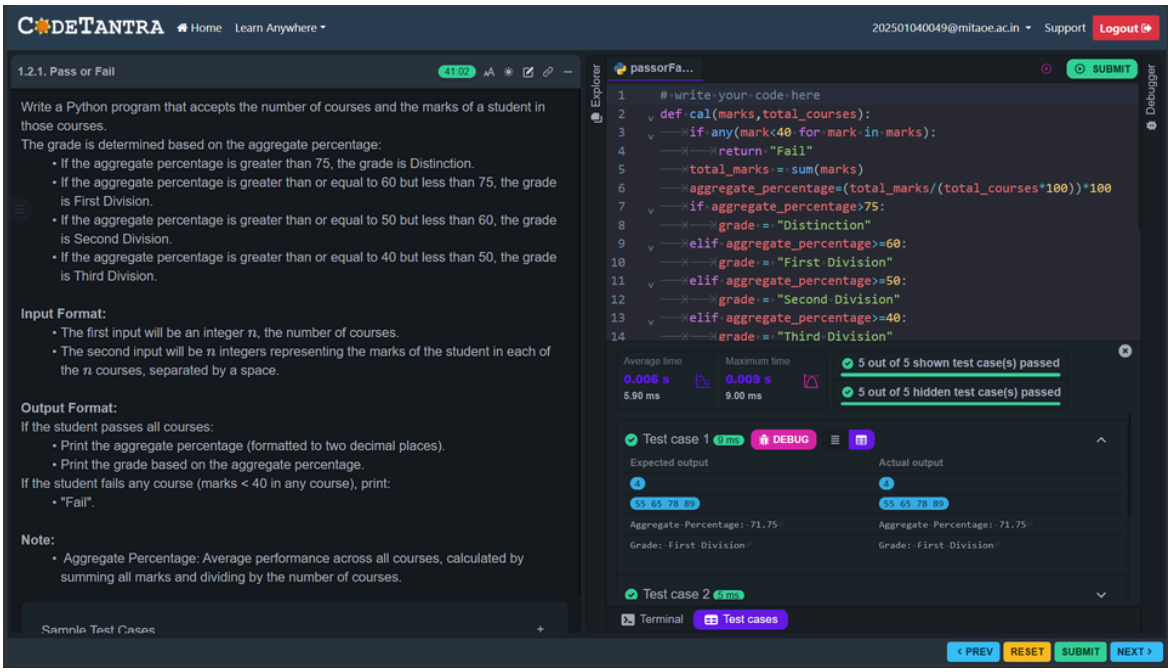
If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
        if any(mark<40 for mark in marks):
                return "Fail"
        total_marks = sum(marks)
        aggregate_percentage = ((total_marks)/(total_courses*100))*100
        if aggregate_percentage > 75:
                grade = "Distinction"
        elif aggregate_percentage >= 60:
                grade = "First Division"
        elif aggregate_percentage >= 50:
                grade = "Second Division"
        elif aggregate_percentage >= 40:
                grade = "Third Division"
        return (aggregate_percentage, grade)
num_courses = int(input())
marks = list(map(int,input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
        print("Fail")
else:
        aggregate_percentage, grade = result
        print("Aggregate Percentage:",f"{aggregate_percentage:.2f}")
        print(f"Grade: {grade}")
```

Output:



1.2.2 Fibonacci Series

ProblemStatement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

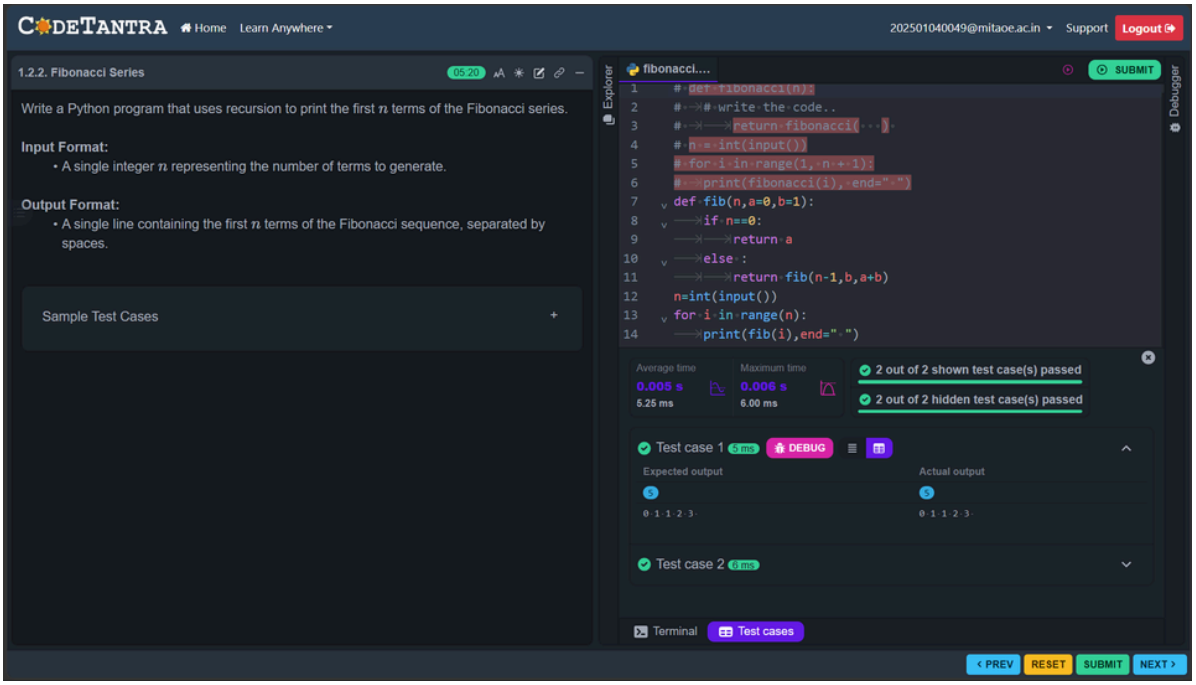
- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):
    if n==0:
        return a
    else:
        return fibonacci(n-1,b,a+b)

n=int(input())
for i in range(n):
    print(fibonacci(i) ,end=" ")
```

Output:



1.2.3 Pattern-1

Problem Statement:
Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

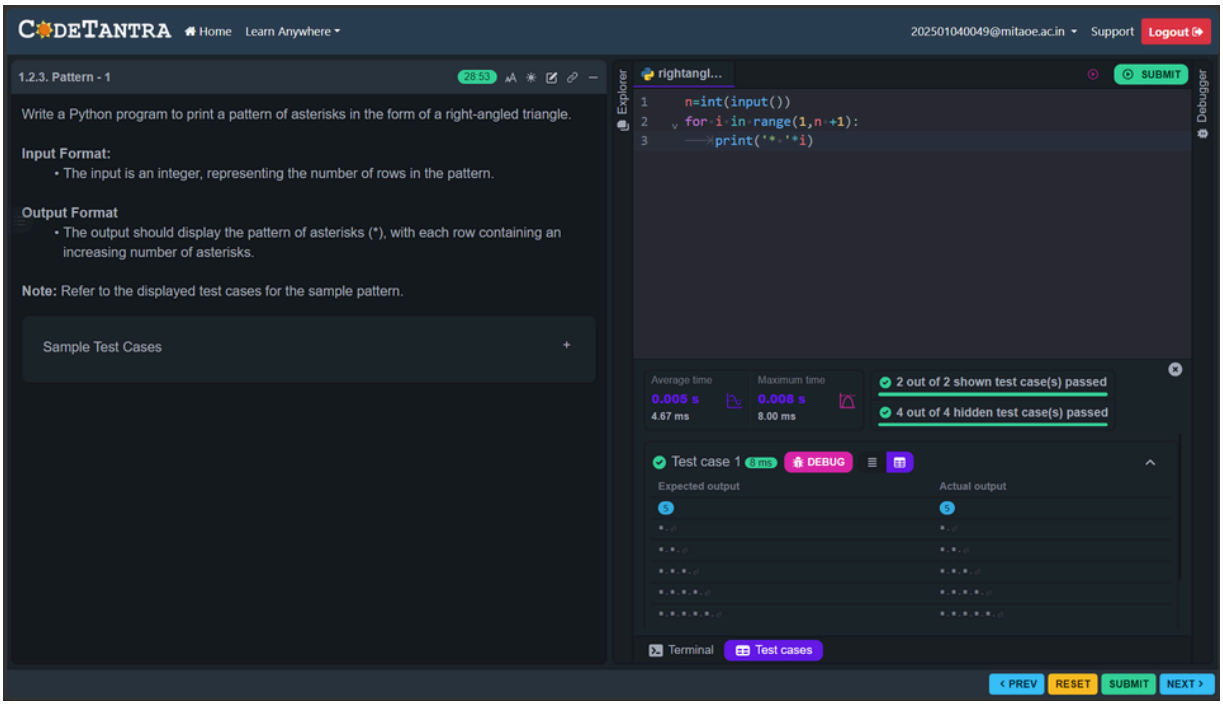
Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
        for j in range(i):
                print("* ",end = "")
        print()
```

Output:



1.2.4 Pattern-2

Problem Statement:
Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

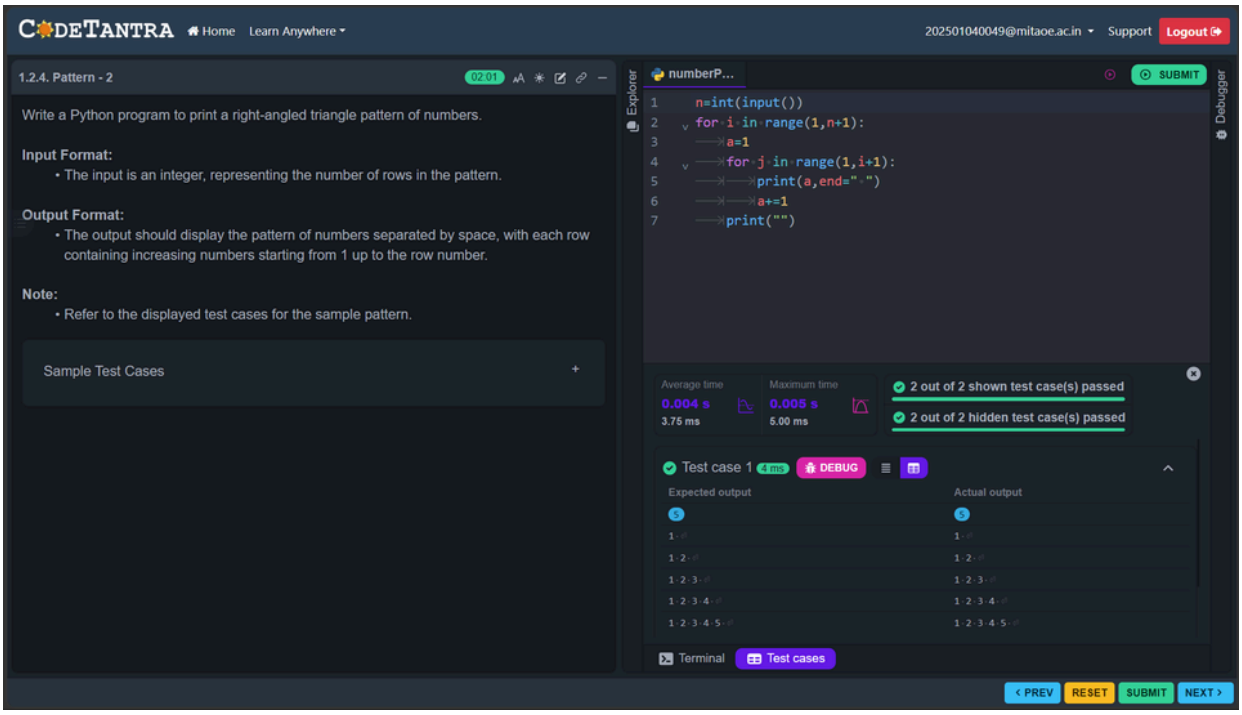
Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
        for j in range(1, i+1):
                print(j,end=" ")
        print()
```

Output:


CODETANTRA
Home
Learn Anywhere
202501040049@mitaoe.ac.in
Support
Logout
1.2.4. Pattern - 2
02:01
Write a Python program to print a right-angled triangle pattern of numbers.
Input Format:
• The input is an integer, representing the number of rows in the pattern.
Output Format:
• The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.
Note:
• Refer to the displayed test cases for the sample pattern.
Sample Test Cases
Explorer
numberP...
SUBMIT
Debugger
n=int(input())
for i in range(1,n+1):
a=1
for j in range(1,i+1):
print(a,end=" ")
a+=1
print("")
Average time
0.004 s
3.75 ms
Maximum time
0.005 s
5.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
4 ms
DEBUG
Expected output
Actual output
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›